TABLE OF CONTENTS

How to Navigate Without Context with GoRouter and Riverpod in Flutter

#dart #flutter #gorouter #navigation #riverpod

Andrea Bizzotto
Updated Feb 5, 2023 | 12 min read

Have you ever needed to pop the current route or navigate to a new screen based on some **conditional logic** or **after running some asynchronous code**?

Maybe you received a notification and need to navigate to page A or page B depending on the message payload.

Or maybe you need to submit a form and only pop the current route if the data was written without errors, just like in this example:

```
// some button callback
onPressed: () async {
  try {
    // get the database from a provider (using Riverpod)
    final database = ref.read(databaseProvider);
    // do some async work
    await database.submit(someFormData);
    // on success, pop the route if the widget is still mounted
    if (context.mounted) {
      context.pop();
    }
  } catch (e, st) {
    // TODO: show alert error
  }
}
```

As we can see, the call to `context.pop()` only takes place **conditionally** and **after an asynchronous gap**, based on some business logic.

But where should the business logic and routing code belong?

If we keep everything inside a widget callback (like in the example above), our logic gets all mixed up with the UI code and becomes hard to test. 😢

If we move the business logic to a separate class, we establish a **good separation of concerns**. But how can we **navigate without context** outside our widgets?

I'm glad you asked. 😊

For in this article, we'll explore three different navigation techniques, along with their tradeoffs:

1. Passing the `BuildContext` as an argument
2. Using an `onSuccess` callback
3. Navigate without context using `ref.read(goRouterProvider)`

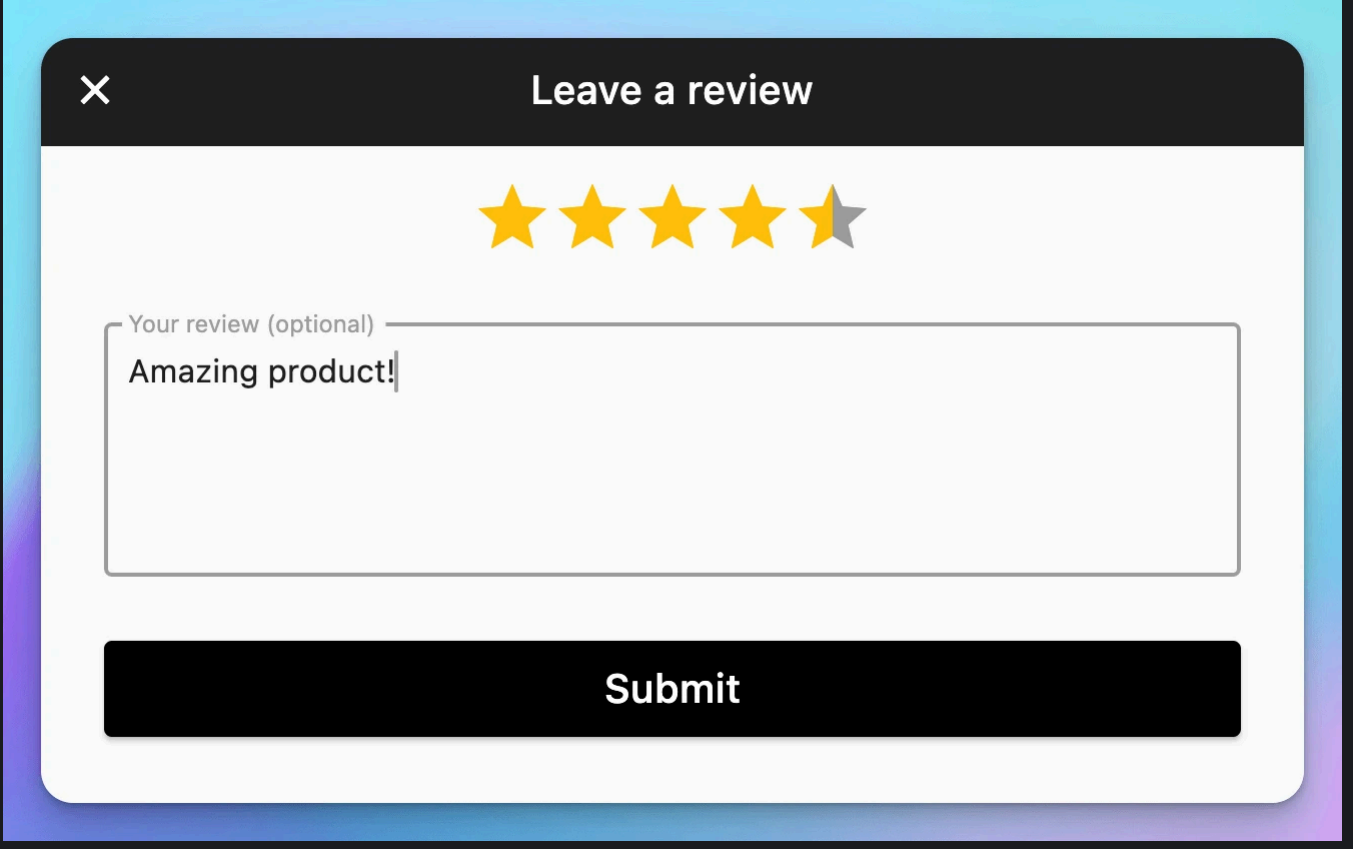
We will use GoRouter for navigation and the `AsyncNotifier` class from the Riverpod package, but the same principles apply if you use `ChangeNotifier` or `Cubit`, or a different navigation package altogether.

By the end, you'll know a few more tricks for writing **maintainable routing code**.

> A reliable way to achieve context-less navigation is by declaring a global `navigatorKey` as explained in this StackOverflow answer. This works well for apps that still use Navigator 1.0. But in this article, we'll focus on a more modern approach for apps built with GoRouter and Riverpod.

Example: Leave Review UI

Suppose we have a form that we can use to submit a review:



Example UI to leave a review for a given product

Here's what the leave review flow may look like:

- from a product page, we click on a button and navigate to the "Leave a review" page
- we select a rating score (1 to 5) and optionally leave a review comment
- when we press the submit button, we attempt to write the data to the database, and **only if** the operation succeeds, we pop back to the previous page

Nothing surprising here - just a classic CRUD example.

So let's see how we can implement this. 👇

Implementation with a widget class

As a starting point, let's consider a `LeaveReviewForm` widget class that we could use to represent the UI above:

```
class LeaveReviewForm extends ConsumerStatefulWidget {
  const LeaveReviewForm({super.key, required this.productId});
  final ProductID productId;

  @override
  ConsumerState<LeaveReviewForm> createState() =>
```

```
_LeaveReviewFormState();
}

class _LeaveReviewFormState extends ConsumerState<LeaveReviewForm> {
  // a state variable that will be updated when we change the rating
score
  double _rating = 0;
  // a controller that will be used to get the text from the comment
field
  final _controller = TextEditingController();

  @override
  Widget build(BuildContext context) {
    // return a form with a rating bar, a text field, and a submit button
  }
}
```

And suppose we have a `_submitReview` method that is called from the `onPressed` callback of the submit button:

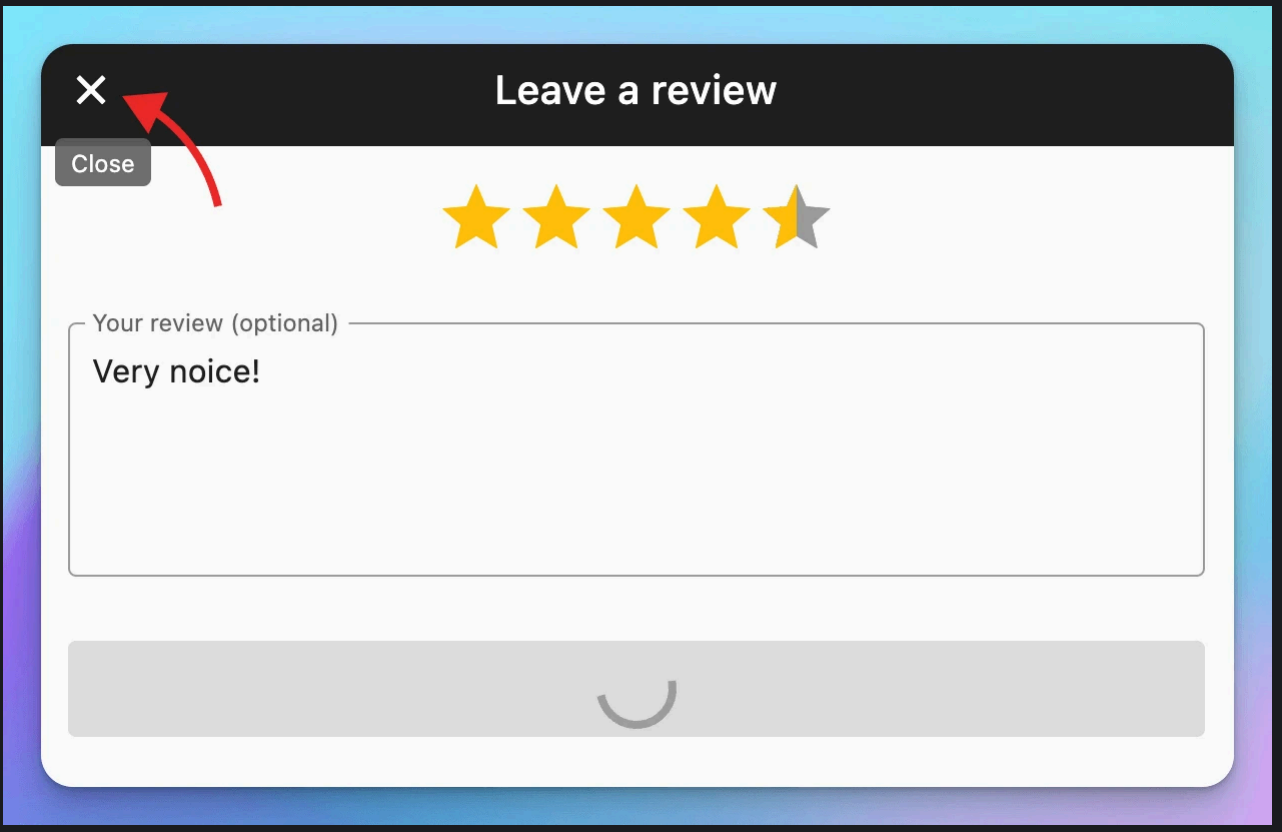
```
class _LeaveReviewFormState extends ConsumerState<LeaveReviewForm> {

  // called from a button callback
  Future<void> _submitReview() async {
    try {
      // create a review object with the rating and comment
      final review = Review(rating: _rating, comment: _controller.text);
      // obtain the review service from a Riverpod provider
      final reviewsService = ref.read(reviewsServiceProvider);
      // use it to submit the review (productId is a property from the
widget class)
      await reviewsService.submitReview(productId: widget.productId,
review: review);
      // the widget is still mounted after the asynchronous operation
      if (context.mounted) {
        // pop the current route (uses GoRouter extension)
        context.pop();
      }
    } catch (e, st) {
      // TODO: show alert error
    }
  }
}
```

A few things to note:

- The method is **asynchronous** since we have to `await` for the review to be submitted before doing anything else
- If the operation fails, we use the `catch` block to show an alert error and **remain on the same page**
- If the operation succeeds, we pop the route only **if the widget is still mounted**

The last point is important because if the user closes the page **before the asynchronous operation completes**, the current route will change, and we don't want to pop it (again!):



What happens if we close the page before the operation has completed?

> Since Flutter 3.7, the `BuildContext` type contains a `mounted` property that we can use to check if the widget is still in the widget tree. This is useful when we want to do something with the context after an asynchronous gap. To learn more, read: Using context.mounted in Flutter 3.7.

The code above works, but keeping our business logic in the widget class is not ideal.

So let's improve this by using an `AsyncNotifier` instead.

If you're not familiar with how `AsyncNotifier` works, read: How to use Notifier and AsyncNotifier with the new Flutter Riverpod Generator

Implementation with AsyncNotifier

As the next step, we can move all our business logic into a new `LeaveReviewController` class that extends `AutoDisposeAsyncNotifier`:

```
// a controller class that will hold all our logic
class LeaveReviewController extends AutoDisposeAsyncNotifier<void> {
  @override
  FutureOr<void> build() {
    // nothing to do
  }

  Future<void> submitReview({
    required ProductID productId,
    required double rating,
    required String comment,
  }) async {
    // create a review object with the rating and comment
    final review = Review(rating: rating, comment: comment);
    // obtain the review service from a Riverpod provider
    final reviewsService = ref.read(reviewsServiceProvider);
    // set the widget state to loading
    state = const AsyncLoading();
    // submit the review and update the state when done
    state = await AsyncValue.guard(() =>
        reviewsService.submitReview(productId: productId, review:
review));
    // TODO: check for mounted
    if (state.hasError == false) {
      // TODO: pop route
    }
  }
}

// the corresponding provider
final leaveReviewControllerProvider =
    AutoDisposeAsyncNotifierProvider<LeaveReviewController, void>(
        LeaveReviewController.new);
```

A few notes:

- We extend from `AutoDisposeAsyncNotifier` rather than `AsyncNotifier`, and the provider type is `AutoDisposeAsyncNotifierProvider`. This ensures that **the controller is disposed as soon as the widget itself is unmounted**.
- In the `submitReview` method, I have replaced the old `try`/`catch` block with a call to `AsyncValue.guard` (*this is an optional step*).
- The `state` is set twice (first with `AsyncLoading`, then with the result of `AsyncValue.guard`). This is so we can handle loading and error states in the widget.
- There are some TODOs about "mounted" and "pop route" since we don't have a `context` inside the controller, and we can't move all the widget code as-is.

> If you're not familiar with `AsyncValue.guard`, read: Use AsyncValue.guard rather than try/catch inside your AsyncNotifier subclasses

We'll deal with the TODOs shortly. But note how the widget code is already much simpler since we can remove the old `_submitReview` method and do this instead:

```
onPressed: () => ref.read(leaveReviewControllerProvider.notifier)
  .submitReview(
    productId: widget.productId,
    rating: _rating, // get the rating score from a local state variable
    comment: _controller.text, // get the text from the
TextEditingController
  ),
```

Taking care of Navigation

As a reminder, we want to pop the current route if `reviewsService.submitReview` completes successfully.

And as we have seen, inside our widget, we were using the `context` to do this:

```
// the widget is still mounted after the asynchronous operation
if (context.mounted) {
  // pop the current route (uses GoRouter extension)
  context.pop();
}
```

But now that our logic lives inside the `LeaveReviewController`, what should we do?

Here are some options:

1. Pass the BuildContext as an argument

By passing `BuildContext` to the `submitReview` method, we end up with something like this:

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

class LeaveReviewController extends AutoDisposeAsyncNotifier<void> {
  // @override build method

  Future<void> submitReview({
    required ProductID productId,
    required double rating,
    required String comment,
    required BuildContext context,
  }) {
    // all the previous logic here
    state = await AsyncValue.guard(...);
    // then do this:
    if (state.hasError == false) {
      if (context.mounted) {
        // pop the current route (uses GoRouter extension)
        context.pop();
      }
    }
  }
}
```

This is **bad** since we don't want our controller to depend on the UI (*note how we're forced to import `material.dart` if use `BuildContext`*). And it means we can no longer write unit tests

for it.

2. Use an onSuccess callback

A better solution is to pass a **callback**:

```
class LeaveReviewController extends AutoDisposeAsyncNotifier<void> {
  // @override build method

  Future<void> submitReview({
    required ProductID productId,
    required double rating,
    required String comment,
    required VoidCallback onSuccess,
  }) {
    // all the previous logic here
    state = await AsyncValue.guard(...);
    // then do this:
    if (state.hasError == false) {
      onSuccess();
    }
  }
}
```

Then, from the widget class, we can do this:

```
onPressed: () => ref.read(leaveReviewControllerProvider.notifier)
  .submitReview(
    productId: widget.productId,
    rating: _rating, // get the rating score from a local state variable
    comment: _controller.text, // get the text from the
TextEditingController
    onSuccess: context.pop, // pop using GoRouter extension
  ),
```

With this setup, the `LeaveReviewController` no longer depends on `material.dart`. And we can update any unit tests to check whether the `onSuccess` callback is called.

This is a good step forward.

But this solution is **less clear** since our business logic and the navigation code **no longer belong together**. In other words, we are forced to look at the controller and widget code **separately** to really understand what's going on.

Let's see if we can do better. 👇

3. Navigate without context using ref.read(goRouterProvider)

When writing Flutter apps, GoRouter and Riverpod are a great combo that can unlock some cool tricks! 🚀

One such trick is to create a provider that returns our `GoRouter` instance:

```
final goRouterProvider = Provider<GoRouter>((ref) {
  return GoRouter(...);
});
```

This **makes it easy to access GoRouter anywhere** as long as we have a `ref`. For example, we can configure the `MaterialApp.router` like this:

```
class MyApp extends ConsumerWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final goRouter = ref.watch(goRouterProvider);
    return MaterialApp.router(
      routerConfig: goRouter,
      ...
    );
  }
}
```

It also means that we can update our controller like this:

```
class LeaveReviewController extends AutoDisposeAsyncNotifier<void> {
  // @override build method

  Future<void> submitReview({
```

```
      required ProductID productId,
      required double rating,
      required String comment,
    }) {
      // all the previous logic here
      state = await AsyncValue.guard(...);
      // then do this:
      if (state.hasError == false) {
        // get the GoRouter instance and call pop on it
        ref.read(goRouterProvider).pop();
      }
    }
  }
```

This is great because:

- the controller no longer depends on the `BuildContext`
- we no longer have to use a callback
- the business logic and navigation code **belong together** inside a method that is easy to unit test

What's what I call a win-win-win! 😀

> The implementation above is incomplete, as we still need to deal with the **mounted** state inside the `AsyncNotifier`. To learn more, read: How to Check if an AsyncNotifier is Mounted with Riverpod.

However, keep in mind that context-less navigation should be used carefully. 👇

A word of caution

Once you have a `Provider<GoRouter>` in your app, you may be tempted to use `ref.read(goRouterProvider)` every time you want to navigate without a `BuildContext` (that is, anywhere outside the widgets).

But remember that the `BuildContext` helps us keep track of where we are in the widget tree.

And sometimes, we only want to navigate to a new page if certain conditions are true:

- only pop the current route if the widget is still mounted (just like in the example above)

- only handle a dynamic link and navigate to a new route if the user is not currently filling a form (to prevent data loss)

In these cases, we should consider what page is currently visible and take the application state into account, so we only navigate when appropriate.

> Unless you have a good reason to do so, consider using context-less navigation only from controller classes in the presentation layer.

With that said, let's do a summary of what we have learned so far.

Conclusion

We started with some example code for submitting a form and popping the current route inside a widget class.

Then, we learned how to move all the business logic to an `AsyncNotifier` subclass for better separation of concerns.

And we've explored three ways to perform navigation inside `AsyncNotifier` subclasses:

1. Passing the `BuildContext` as an argument
2. Using an `onSuccess` callback
3. Using `ref.read(goRouterProvider)`

As we have seen, the third option is best because the notifier no longer depends on the `BuildContext` or a callback argument. And the business logic and navigation code **belong together** inside a method that is easy to unit test.

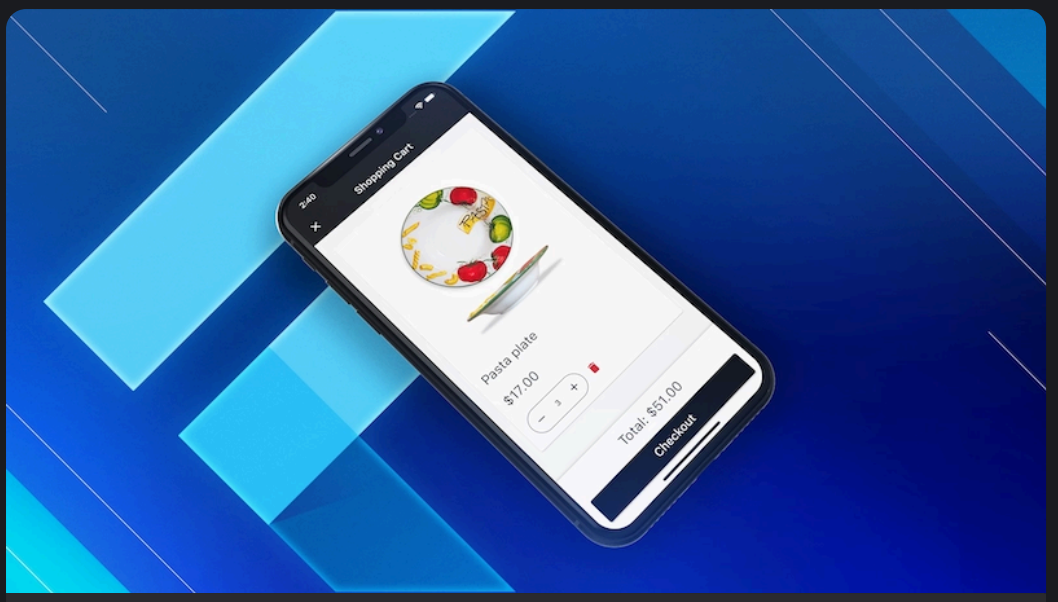
In practice, there are other scenarios where you may need to navigate without context, such as when you receive a notification and need to navigate to different pages depending on the message payload. In these cases, you can also call `ref.read(goRouterProvider)` and use it as needed.

This technique will help you write **more maintainable and testable routing code** as long as you know what you're doing and don't get carried away. So feel free to use it in your Flutter apps. 👍

And if you want to go even more in-depth, check out my latest Flutter course, where you'll learn how to build a complete eCommerce app using GoRouter and Riverpod. 👇

Flutter Foundations Course Now Available

I launched a brand new course that covers state management with Riverpod in great depth, along with other important topics like app architecture, routing, testing, and much more:



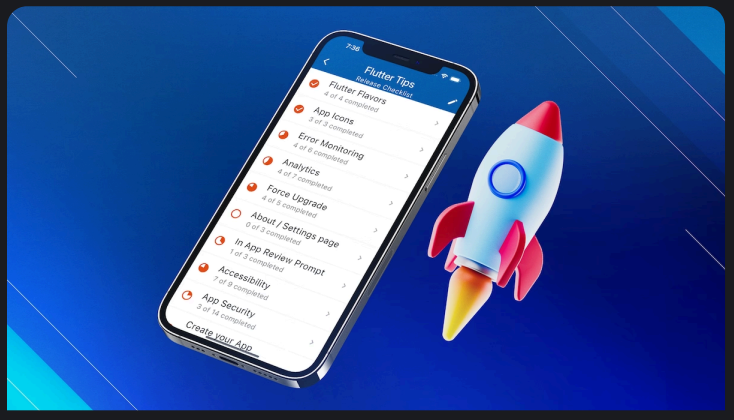
Flutter Foundations Course

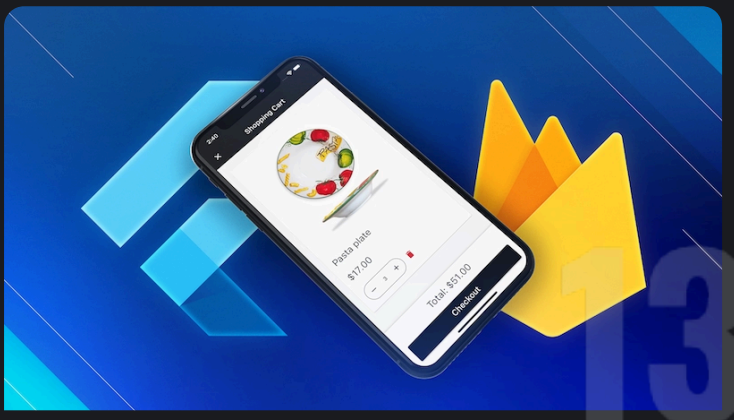
INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

Invest in yourself with my high-quality Flutter courses.





Flutter In Production

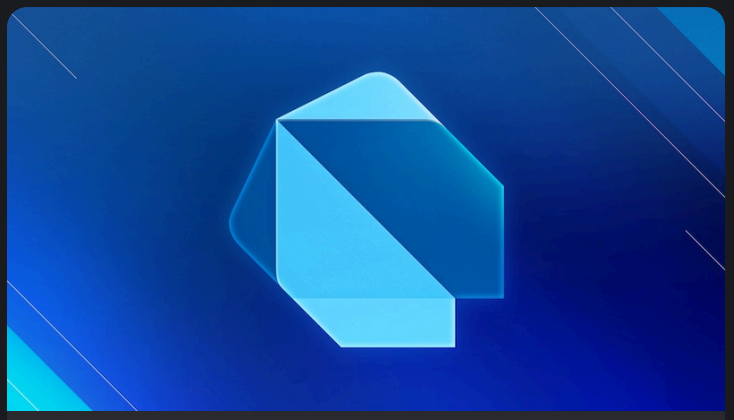
INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀

Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase.



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.

Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.

CODE WITH ANDREA

Contact

Twitter

Discord

GitHub

RSS

Meta

Privacy Policy

Terms of Use